

An Extensive Investigation of Strategies in Baseball

Baseball
20251404

1. Introduction

At numerous points throughout the course of a baseball game, a team's manager must choose an action from a set of multiple possible actions. For example, before every at-bat they must decide whether to stick with the current batter or replace them with a pinch hitter. They must also decide when to relieve their current pitcher and which reliever to use. These decisions can have a significant effect on the outcome of a game and are often heavily scrutinized and criticized when they seemingly backfire, like Kevin Cash's decision to relieve a hot-handed Blake Snell in Game Six of the 2020 World Series [23] or Aaron Boone's decision to use Nestor Cortes against Freddie Freeman in Game One of the 2024 World Series [16].

In this paper we develop and evaluate a number of tools to aid managers in making optimal decisions. We will refer to these tools as our AI managers. In Section 3, we establish a "state of the art" AI manager by showing that the MLB managers who more closely followed the strategies defined by the AI won a statistically significant majority of the time. We call this AI manager the Lindsey strategy since it follows from George R. Lindsey's work in [15]. Then in Section 4, we assess a number of alternatives to the Lindsey strategy in a tournament of simulated games between the AI managers. Finally, in Section 5, we use our champion AI manager to reassess Boone and Cash's infamous decisions.

To begin, we model a baseball game as a stochastic, zero-sum, perfect-information, extensive-form game played between two managers who each want to choose the actions that maximize their own team's expected win probability. Making use of notation from [20] and [21], the game is defined by $P, S, Z, A, \rho, \chi, \sigma$, and u where

- $P = \{\text{home manager, away manager, chance}\}$ is the set of players.
- S is the state space. A state $s \in S$ is characterized by the inning, score, baserunners, outs, the lineups of both teams, the available bench players on both teams, the available relief pitchers on both teams, the current pitcher and batter for both teams, the number of batters each team's pitcher has faced, and whether it's an offensive state, defensive state, chance state, or terminal state, which determines which player acts in that state.
- $Z \subset S$ is the set of terminal states.
- A is the set of all possible actions. We consider only matchup-level actions, such as which reliever to bring into the game, as opposed to pitch-level actions such as whether to throw a fastball or slider on the next pitch or whether to attempt a bunt or stolen base.
- $\rho: S \rightarrow P$ is the player function that assigns a player to each state. This is the player that must take an action in that state.
- $\chi: S \times P \rightarrow 2^A$ is the action function that defines a set of admissible actions for a player in a given state.
- $\sigma: S \times A \rightarrow S$ is the transition model that defines the state that results from an action in a given state.

- $u: Z \rightarrow \{0, 1\}$ is the utility function. Since this is a zero-sum game, our utility function maps to 1 if the home manager wins in the terminal state and 0 if the away manager wins. Therefore, the home manager's goal is to maximize utility and the away manager's goal is to minimize utility.

The game is depicted using a graph theory tree with three different types of nodes: choice, chance, and terminal nodes. Each node has a unique state. A node is a *terminal node* if its state s is in Z . Choice and chance nodes are assigned a player by the player function ρ . If for a node with state s we have $\rho(s) \in \{\text{home manager, away manager}\}$, then that node is a *choice node*. Otherwise, if $\rho(s) = \text{chance}$, then the node is a *chance node*. Actions are defined for each non-terminal node using the action function χ and they are represented in the game tree by edges. The action function χ considers positional eligibility when defining the action sets, e.g. a manager cannot pinch hit for their only backup catcher if the starting catcher has already been removed from the game. The state transition function σ defines the resulting state when an action/edge is chosen. In choice nodes, the assigned player p chooses an action in $\chi(s, p)$, which is equivalent to choosing the edge to follow in the game tree. In chance nodes, an action is randomly sampled from $\chi(s, \text{chance})$ according to a probability distribution over $\chi(s, \text{chance})$. We use chance nodes to represent plate appearances between a batter and a pitcher, and we define the probability distribution over $\chi(s, \text{chance})$ using a batter and pitcher matchup projection model, which we describe in more detail in Section 2.

The structure of the tree follows a consistent pattern. Before each plate appearance, there is first a batter choice node where the manager on offense chooses to stick with the on-deck batter or to pinch hit. After any batter choice node there is a pitcher choice node where the manager on defense chooses to let the current pitcher pitch to the chosen batter, intentionally walk the chosen batter, or relieve the current pitcher if the pitcher is allowed to be relieved [4]. If they relieve the current pitcher, we allow the offensive team to respond by transitioning back to a batter choice node. If they intentionally walk the chosen batter, we transition to the batter choice node corresponding to the next batter. Otherwise, if they choose to let their pitcher face the chosen batter, we transition to a chance node where we randomly simulate a plate appearance between the chosen batter and pitcher. If the plate appearance does not end the inning or game, we transition to another batter choice node corresponding to the next batter and restart the pattern. If the plate appearance ends the inning, we transition to a defensive substitution choice node where the former offensive manager who is now on defense can make defensive substitutions. If the plate appearance ends the game, we transition to a terminal node and define the game utility using $u(s)$.

Figure 1 depicts a small, toy example of our game tree. Suppose there are two outs and bases loaded in the bottom of the ninth inning of a tie game. Suppose also that we know that the home team will lose if the game goes to extra innings, so if the next batter makes an out they lose. The Δ nodes are the maximizing home manager's nodes, the ∇ nodes are the minimizing away manager's nodes, and the $\bigcirc$ nodes are the chance nodes. Terminal nodes are represented by their utility in $\{0, 1\}$. The next batter due up for the home team is Mikey Thomas, who is known to strike out in 70% of his plate appearances and homer in the other 30%. The home manager also has one player left on their bench, Pablo Sanchez, who homers in 70% of his plate appearances and strikes out the other 30%. The away team is on their last pitcher, so their only possible actions are to pitch to the chosen batter or to intentionally walk the chosen batter (IBB). It is very clear from this example that the away manager should not choose to intentionally walk either possible batter, and the home manager should choose to pinch hit Pablo Sanchez for Mikey Thomas, which gives them an expected utility (win probability) of 0.7 if the away manager chooses not to intentionally walk.

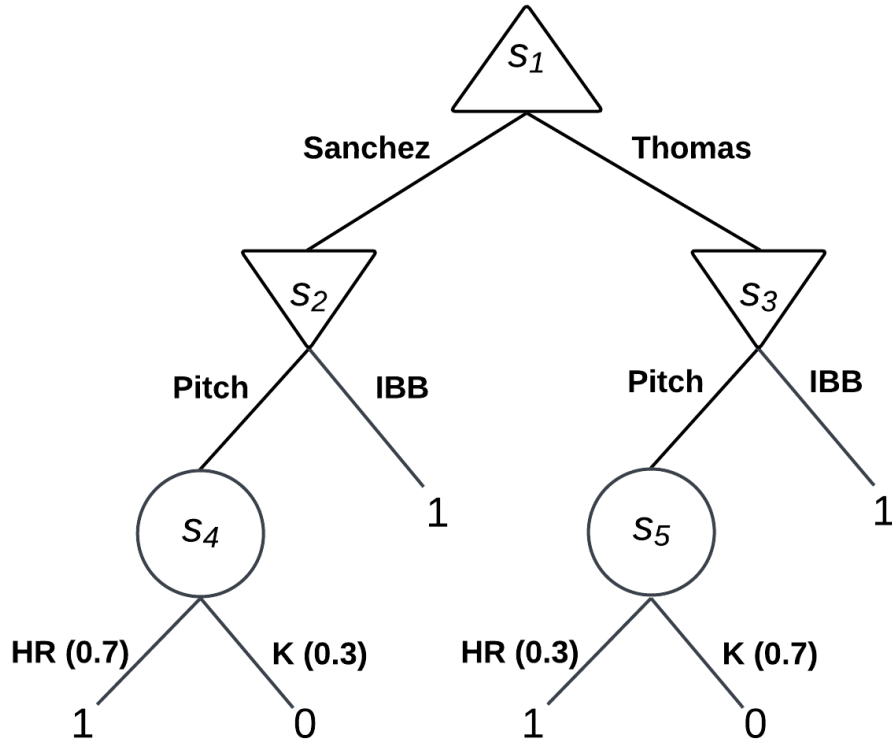


Figure 1: Example game tree. $\triangle$ nodes are the maximizing player's nodes, ∇ nodes are the minimizing player's nodes, and $\bigcirc$ nodes are the chance nodes. It is clear that the optimal strategy for the maximizing home manager is to bat with Sanchez instead of Thomas. The optimal strategy for the minimizing away manager is to pitch to both batters. The expected utility (home team win probability) if both players follow the optimal strategy is 0.7

A strategy, which we will denote $\pi: S \rightarrow A$, for a player in an extensive-form game is a function that returns an action for each of the player's choice nodes. In the previous example, we determined that the optimal strategy for the home manager was $\pi_{\text{home}}(s_1) = \text{Sanchez}$, and the optimal strategy for the away manager was $\pi_{\text{away}}(s_2) = \pi_{\text{away}}(s_3) = \text{Pitch}$. These strategies are best responses to each other, meaning that the strategy that minimizes utility when the home manager plays strategy π_{home} is π_{away} , and the strategy that maximizes utility when the away manager plays π_{away} is π_{home} . A subgame-perfect equilibrium (SPE) is a pair of strategies for the home and away manager that are best responses to each other in every subtree of the larger game tree, including the full game tree itself [21]. For example, $(\pi_{\text{home}}, \pi_{\text{away}})$ is a SPE in the previous example.

Since a truly optimal baseball manager will always use a best response strategy against their opponent's actions, then we can create the best possible AI manager by solving for a SPE in our extensive-form baseball game. The challenge in finding a SPE for a baseball game is that $|S| = \infty$. There is always a nonzero probability of the next batter not making an out, which means there is always a nonzero probability of scoring n runs in an inning for any $n \in \mathbb{N}$. Likewise, there is always a nonzero probability of the next batter making an out, which means a game could end in a 0-0 tie and then go into extra innings for any number of $n \in \mathbb{N}$ innings. We therefore make the following

assumptions: a team will win when they go up by a large number of runs in an inning and all extra innings are equivalent to the tenth. These constraints ensure $|S| < \infty$, but we still estimate our state space to be larger than a game like Heads-Up Limit Hold' em, which took over two months to solve for an equilibrium strategy [8]. Given that MLB teams play almost every day of the summer and given that team lineups are usually not known until hours or even minutes before a game, our AI managers need to be able to calculate their solutions in hours or minutes. Thus, our AI managers represent a variety of methods to efficiently approximate the optimal strategy. Our results show that exploiting extensive-form games to reduce the state space yields better decisions in a reasonable amount of time than exist in the current literature. We also show that an implementation of MCTS does not perform well against these alternatives, suggesting naïve approaches may not be sufficient for in-game use.

2. Matchup Model

The baserunner-out state is the part of the game state s that describes the runners on base and the number of outs. We use the notation $[000\ 0]$ for baserunner-out states, where the first three characters correspond to there being a runner on first, second, and third base respectively, and the fourth character is the number of outs. For example, $[000\ 0]$ means nobody is on base and nobody is out, $[010\ 1]$ means there's a runner on second and one out, and $[111\ 2]$ is bases loaded and two outs. A plate appearance can be thought of as a transition from one of 24 possible starting baserunner-out states to one of 25 possible ending baserunner-out states, which are the 24 starting states plus the inning-ending three out state. Thus, the goal of our matchup model is to learn a 25×24 batter and pitcher-specific transition matrix whose $(i, j)^{\text{th}}$ entry gives the probability of the baserunner-out state transitioning from state j to state i when the batter faces the pitcher in state j . Then for a chance node with state s and baserunner-out state j , we define the distribution over $\chi(s, \text{chance})$ as the j^{th} column of the transition matrix corresponding to the batter and pitcher in state s .

We define a batter and pitcher-specific transition matrix as

$$\sum_{o \in O} P(o | \text{batter}, \text{pitcher}) T_o$$

where O is the set of plate appearance outcomes, i.e. $O = \{\text{ball-in-play out, single, double, triple, home run, batter hit by pitch, walk, strikeout}\}$. Note, we count fielders' choices, fielding errors, double plays, triple plays, and sacrifice hits all as ball-in-play outs, meaning a batter may get on base even if the plate appearance ends with the ball-in-play out outcome. T_o is the outcome-specific baserunner-out transition matrix. We assume that triples, home runs, hit by pitches, walks, and strikeouts have deterministic transitions: home runs and triples clear the bases, triples add a runner to third base, strikeouts increase outs by one, and walks and hit by pitches put a runner on first and move any necessary runners over a base. For outs, singles, and doubles, we calculate the transition probabilities using the historical transition rates in MLB play-by-play data from 2019 to 2023 collected from retrosheet [1, 6, 17].

To estimate $P(o | \text{batter}, \text{pitcher})$, we used MLB plate appearance outcome data [2, 3, 14] and the Marcel projection model [22] to project the rate of each outcome per plate appearance for each batter and pitcher broken down by opponent handedness. For example, for Aaron Judge we

projected his outs, singles, doubles, triples, home runs, hit by pitches, walks, and strikeouts per plate appearance against both right and left-handed pitchers. We then converted those rates into log odds and learned a multinomial logistic regression model with the batter log odds of each outcome and the pitcher log odds of each outcome as inputs. $P(o | \text{batter, pitcher})$ is then given by the outputs of the multinomial logistic regression model.

Despite the simplicity of the Marcel projection model, which the creator of the model described as “The most basic forecasting system you can have, that uses as little intelligence as possible” [22], our model of $P(o | \text{batter, pitcher})$ makes well calibrated predictions on a validation dataset, as evidenced by Figure 2. Additionally, in Section 3, we will show that managers who strategize

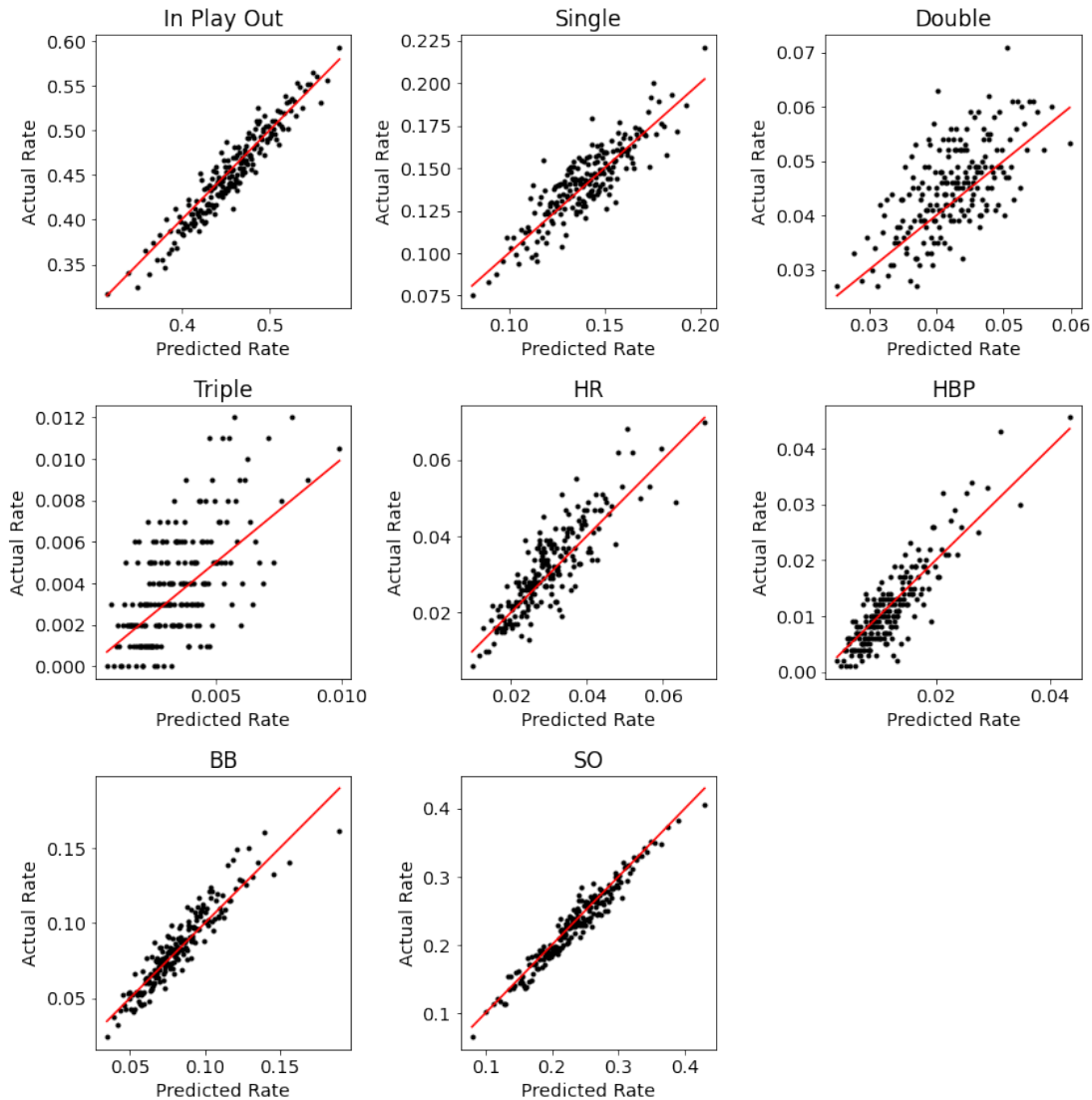


Figure 2: Probability calibration plots on a holdout validation dataset for our multinomial logistic regression model of $P(o | \text{batter, pitcher})$.

according to the chance node transition probabilities defined by our simple matchup model can win a statistically significant majority of the time. However, there are at least two potential issues with using this simple matchup model to learn manager strategies. First, the model does not consider the effect of fielder skill on the distribution over plate appearance outcomes. Thus, the actions in our defensive substitution choice nodes are judged based on how the new lineup will perform on offense in the next half inning rather than on how well they will perform defensively. Second, the model does not consider how a pitcher will fatigue throughout the course of the game, which means there is no difference in our game model between a pitcher who just entered the game with a pitch count of zero and the same pitcher after seven innings with a pitch count over 100. We intend to address these issues with a better matchup model in future work. However, batter versus pitcher matchup modeling is a thoroughly researched problem [7, 9 – 12, 18]. Additionally, MLB team analysts create their own proprietary matchup models. The purpose of this paper was not to create a state of the art matchup model. Instead, the purpose was to optimize managerial decisions given a choice of one of the many possible choices of matchup models. Any choice of matchup model can be used in our extensive-form game model.

3. The Lindsey Strategy

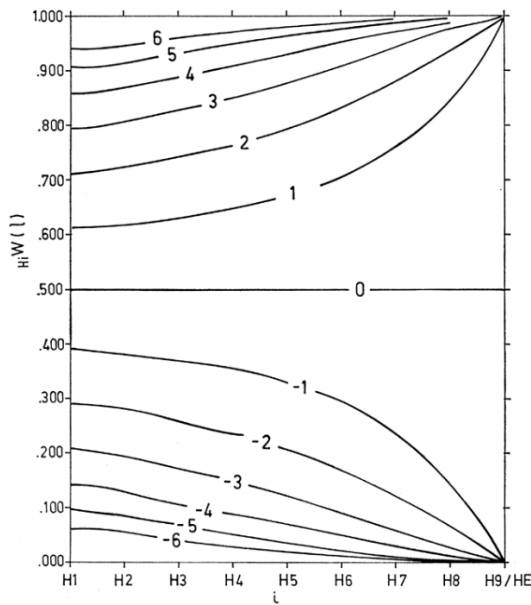
Our first AI manager, the Lindsey strategy, is a direct extension of George R. Lindsey's work in *An Investigation of Strategies in Baseball* [15]. Lindsey used data that tracked the inning-by-inning score of a large number of baseball games to estimate the probability that the home team wins at the end of the i^{th} inning if they have a lead of l runs, which he denoted $_{Hi}W(l)$. Figure 3a, which comes straight from Lindsey's paper, shows a contour plot of $_{Hi}W(l)$. We learned a modern version of $_{Hi}W(l)$ by fitting an xgboost classifier to predict if the home team wins using the score and the half inning as input. We trained the classifier using the retrosheet play-by-play data from 2019 to 2023 [1, 6, 17]. Our version of Lindsey's contour plot is depicted next to Lindsey's in Figure 3b.

Lindsey then used data that tracked the number of runs that scored for the remainder of a half inning from a given baserunner-out state to empirically estimate the probability distribution $P(r | b)$, where r is the runs that score for the remainder of the inning and b is one of the 24 possible starting baserunner-out states. Lindsey then defined an estimate of the home team's win probability given a lead l , inning i , and baserunner-out state b as

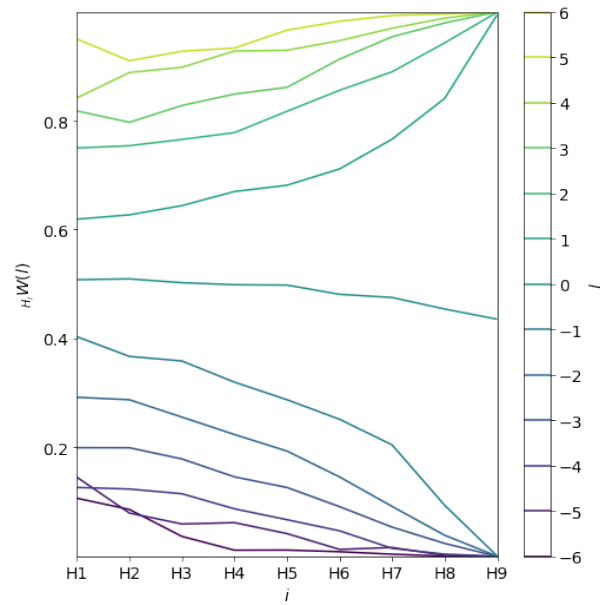
$$\Omega(i, l, b) = \sum_{r=0}^{\infty} P(r | b)_{Hi}W(l + r). \quad (1)$$

We call this value the Lindsey win probability. Note, we estimated a modern version of $P(r | b)$ once again using the retrosheet play-by-play data from 2019 to 2023 [1, 6, 17]. Since we estimate that probability distribution empirically, and since no team has ever scored infinite runs, the infinite sum is not an issue.

Lindsey used Ω to evaluate the effects of certain in-game decisions on a team's win probability. For example, we know deterministically the baserunner-out state b' that results from an intentional walk in starting baserunner-out state b . Thus, we can estimate if an intentional walk improves the home team's win probability by calculating $\Omega(i, l, b') - \Omega(i, l, b)$. If that value is positive, the home team should intentionally walk, otherwise they should not. This idea leads us to our first AI manager, called the Lindsey strategy. Since the states in our extensive-form game keep track of the



(a) George Lindsey's contour plot [15]



(b) Our version of Lindsey's contour plot

Figure 3: The contour plots showing the home team win probability $H_i W(l)$ by inning i and lead l . The x-axes give the inning and the y-axes give the win probabilities. The contours are the home team's lead. Note the inning labels are H_i for each inning i to signify that we are referring to the home half of the inning.

inning, score, and baserunner-out state, we can define the Lindsey win probability of every node in our game tree. Then instead of searching the entire state space $\mathcal{S}$, which we determined to be infeasible, we can define a subtree rooted at the current choice node and terminating at the conclusion of the next plate appearance. Any leaf node in that subtree that is not already a terminal node is given an estimated home team win probability using Ω . The resulting subtree is small enough that we can easily perform expectiminimax [20] to define a pair of home and away manager strategies that are optimal in that subtree. Those strategies are the Lindsey strategies.

To illustrate the Lindsey strategy, let's revisit our toy example from Figure 1, only this time instead of assuming the home team will lose in extra innings, we define the value of transitioning to extra innings using Ω (Figure 4). If we assume that this small example tree is a subtree of a larger game, then this subtree represents the subtree we use to estimate the Lindsey strategy since it only looks one plate appearance into the future. It is again clear that the optimal Lindsey strategy is for the home manager to pinch hit with Pablo Sanchez and for the away manager to pitch to either batter, rather than intentionally walking them. The expected utility in this subtree if both players follow the Lindsey strategy is $0.7(1) + 0.3(0.56) = 0.868$.

To evaluate the effectiveness of the Lindsey strategy in actual MLB games, we scraped event tracking data from the 2023 MLB season from MLB Gameday [5]. Unlike the play-by-play files used to learn the matchup model, the event tracking data also records managerial decisions like intentional walks and player substitutions. We supplemented the Gameday files with the MLB Stats API, which provided the available players on both teams' benches and in their bullpens on each play [19]. We were then able to define relevant chance and choice nodes for each play in a total of 2296

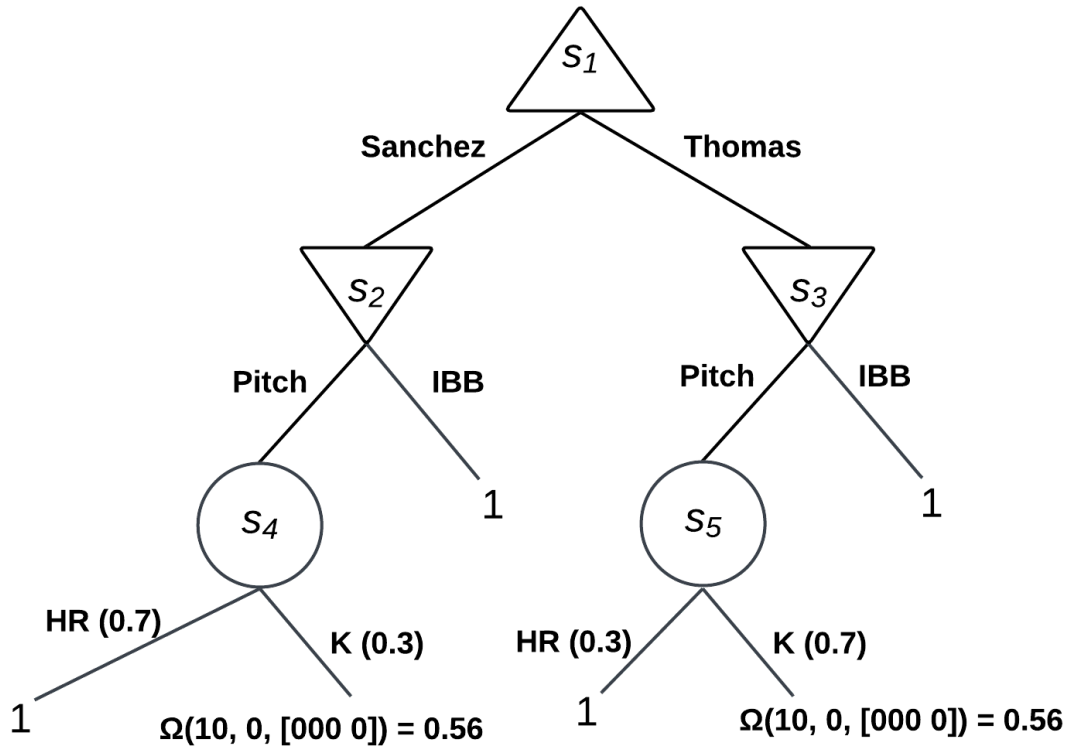


Figure 4: Example of the Lindsey strategy. If either batter, Sanchez or Thomas, strikes out and we transition to a non-terminal node, then we assign a value to that node using the Lindsey win probability function Ω . We can then use expectiminimax in this subtree to find the Lindsey strategy.

MLB games. We ignored defensive substitution nodes since our current matchup model does not consider defensive skill, so we are not equipped to evaluate those decisions. On every other choice node, define the suboptimal cost as the difference in expected Lindsey win probability between the Lindsey strategy and the strategy that managers actually used. For example, in Figure 4, the expected Lindsey win probability for the away manager under the Lindsey strategy is $1 - 0.868 = 0.132$. If the away manager decided to intentionally walk Sanchez instead of pitching to him, their win probability drops to 0, and the suboptimal cost is $0.132 - 0 = 0.132$. In the 2296 games we tested, the manager with lower cumulative suboptimal cost ultimately won 1211 times (52.7%). In other words, the manager that more closely followed the Lindsey strategy won a majority of the time. The probability of observing at least 1211 wins in 2296 games if the win probability is 0.5 is 0.0045, so we conclude that the manager who more closely followed the Lindsey strategy won a statistically significant majority of the time. This suggests that the Lindsey strategy can help managers make good decisions. In the next section, we present four alternative AI managers. Some of these managers take enough time to solve for a strategy that it is impractical to evaluate them on these 2296 games, but having established the Lindsey strategy as a good baseline, we can validate the alternative AI managers by having them play the Lindsey strategy in a large number of simulated games.

4. The AI World Series

We begin this section by introducing our four additional AI managers: deep Lindsey, MCTS, the half game manager, and the pseudo-full game manager. Deep Lindsey is an obvious extension of the Lindsey strategy. Rather than solving for an optimal strategy in the subtree that terminates one plate appearance into the future, it solves for an optimal strategy in the subtree that terminates $n > 1$ plate appearances in the future. For our simulations, we set $n = 3$, and we also terminated the subtree any time a half inning ended. Although a greater search depth would likely return better strategies, it comes with the cost of increased runtime, which is a problem when we simulate thousands of games. Additionally, the search depth we used mirrors the three-batter minimum rule, which requires a pitcher to face three batters or get through an inning before they can be relieved [4]. We can therefore expect deep Lindsey to make better relief pitcher decisions than the Lindsey strategy.

The next AI manager is the Monte Carlo tree search (MCTS) manager, which strategizes using vanilla MCTS [20]. Given a choice node, MCTS recommends an action by performing the following three steps a large number of times in the subtree rooted at the choice node. Note that these steps require a node to keep track of their running total utility and their running total visit count, which we denote $V(s)$ and $N(s)$ for a node with state s .

1. **Selection:** Starting at the root node, iteratively select a child node until one that has never been visited is reached.
2. **Simulation:** Simulate the rest of the game from the unvisited child node according to a simulation function. Our MCTS manager uses a random simulation, meaning actions in subsequent choice nodes are chosen at random.
3. **Backpropogation:** The simulated game's utility is backpropogated up the path that was taken from the root to the unvisited child node. Each node in the path adds the utility to its running total utility and iterates on its visit count.

After running these steps a large number of times, the MCTS manager selects the action that leads to the child s' satisfying

$$s' = \begin{cases} \operatorname{argmax}_{s \in \text{children}} \frac{V(s)}{N(s)} & \text{if home player} \\ \operatorname{argmin}_{s \in \text{children}} \frac{V(s)}{N(s)} & \text{if away player.} \end{cases}$$

The MCTS manager selects children in the selection step according to the upper confidence tree (UCT) algorithm

$$\operatorname{argmax}_{s' \in \text{children}} \frac{V(s')}{N(s')} + 2C \sqrt{\frac{2 \ln N(s)}{N(s)}} \quad (2)$$

where s is the parent node's state and C is the exploration constant [13]. Note when the MCTS manager is playing as the away team, we have to multiply $V(s')$ by -1 . Larger values of C encourage exploration, and smaller values encourage exploitation. We chose $C = \frac{1}{\sqrt{2}}$.

The half game manager significantly reduces the size of the state space S by assuming the goal of each manager is to maximize runs on offense while minimizing runs on defense instead of maximizing their win probability. Under this assumption, a complete baseball game can be split into two half games: one played between the home pitchers and the away offense in the top of each inning and the other played between the away pitchers and the home offense in the bottom of each inning. This is possible because our simple matchup model does not consider the effect of fielder skill on plate appearance outcomes, so the home offense and away pitchers do not affect the runs scored by the away team and the away offense and home pitchers do not affect the runs scored by the home team. The two half games are small enough that we can use expectiminimax to solve for a subgame-perfect equilibrium. However, we define some additional constraints to solve the half games in a couple hours on a single CPU core. They are:

- Six of the starting nine batters from each team must stay in the game while the other three can be substituted.
- Each team has four batters on their bench, giving them a total of 13 batters.
- Each team has five starting pitchers and eight relievers, but only the relievers are allowed to come out of the bullpen.

These constraints align with the 26-man roster limit imposed by MLB throughout most of the regular season and all of the postseason along with the common practice of not pinch hitting for the most talented batters on a team. Once these constraints are applied, the half game manager solves for a subgame-perfect equilibrium in both half games before a game begins and then refers to them in a lookup table during the game to make decisions.

Although the half game's assumption that a manager should play to maximize their runs and minimize their opponent's runs is reasonable in most cases, there are some situations where maximizing/minimizing runs may not be the right idea. For example, Lindsey determined that in the bottom of the ninth inning with the home team down by one run, runners on second and third base, and one out, an intentional walk lowers the home team's Lindsey win probability (1) by 0.041 even though the expected runs that score for the remainder of the half inning increases by 0.082 [15]. Thus, the half game manager may recommend that the away team pitches to the next batter when the optimal decision may be to intentionally walk them. The pseudo-full game manager addresses this issue by adding a chance node between innings in the half games. This chance node represents a probability distribution over the runs that score in the other half of the inning. The probabilities can be specific to the teams playing in the current game, or league averages can be used (which is what we do for this paper). By including this chance node, the pseudo-full game manager keeps track of the scores of both teams, allowing it to use wins instead of runs for the utility without significantly increasing the size of the state space. Then, under the same additional constraints imposed for the half-game manager, the pseudo-full game manager can solve for a subgame-perfect equilibrium in its version of the game in a few hours.

Having defined our four additional AI managers, we can now begin the AI World Series. We divide the AI World Series into two parts: the qualifying round and the final round. For an AI manager to qualify for the final round, they have to beat the baseline Lindsey strategy in their simulated games. For deep Lindsey and MCTS, we simulated 1,000 games against the Lindsey strategy by randomly choosing a lineup from one of the 2296 MLB games we evaluated in the previous section. We had that lineup play a clone of itself to prevent an AI manager from gaining an advantage due to lineup talent. We also randomly decided which AI manager played as the home or the away team in each simulated game to prevent either from getting a consistent home team advantage.

For the half and pseudo-full game managers, since they require additional lineup constraints, we predefined one lineup for each of the 30 MLB teams with appropriate constraints. Unlike the deep Lindsey strategy and MCTS, the half and pseudo-full game managers solve for their strategies before the game begins. Once these strategies are found, we can simulate games much more quickly than deep Lindsey or MCTS. Thus, instead of simulating 1,000 games against the Lindsey strategy, we simulated 2,500 home games and 2,500 away games for each of the 30 predefined lineups. However, we had memory limits when we solved for the half and pseudo-full game strategies as well as time limits. For some of the lineups, the half and pseudo-full game managers either used up all the available memory or timed out, so in the end we simulated 130,000 games between the half game manager and the Lindsey strategy and 80,000 games between the pseudo-full game manager and the Lindsey strategy.

<i>AI Manager</i>	<i>Games</i>	<i>Wins</i>	<i>Losses</i>	<i>Pct.</i>	<i>p value</i>
<i>Deep Lindsey</i>	1,000	533	467	0.533	0.020
<i>MCTS</i>	1,000	483	517	0.483	0.148
<i>Half Game</i>	130,000	73,208	56,792	0.563	0.00
<i>Pseudo-Full Game</i>	80,000	45,668	34,332	0.571	0.00

Table 1: AI managers' win-loss records against the baseline Lindsey strategy in the qualifying round. All but MCTS qualified for the final round.

Table 1 shows the results of the qualifying round. The p value we report is the probability of observing at least that many wins or losses in the number of games if we assume the true win probability is 0.5. Only the MCTS manager failed to qualify for the final round with a win rate of 0.483 against the baseline Lindsey strategy. It is possible that more search iterations, a more finely-tuned exploration constant C in (2), or a better than random simulation function in the simulation step would have given the MCTS manager the win against the baseline Lindsey strategy. However, given the success of the other three AI managers against the Lindsey strategy, we felt comfortable retiring the MCTS manager. The pseudo-full game manager had the best qualifying round, winning 57.1% of its games against Lindsey. The half game manager was a close second with a win rate of 56.3%, and the deep Lindsey manager was the lowest rated qualifier with a win rate of 53.3%.

For the final round of the AI World Series, we simply had the three finalists play simulated games against each other. Since any matchup between two of the three finalists involves either the half or pseudo-full game manager, we limited the lineups to the 30 predefined lineups with the additional constraints. The state space in games involving these lineups is smaller than the games from deep Lindsey's qualifying round, which means we could increase deep Lindsey's search depth without drastically increasing runtime. If the half/pseudo-full game manager took less than 90 minutes to solve for their strategy, we increased the deep Lindsey search depth to the full half inning of plate appearances. If the half/pseudo-full game manager took between 90 and 180 minutes to solve for their strategy, we increased the deep Lindsey search depth to eight plate appearances. Otherwise, we increased the deep Lindsey search depth to six plate appearances. In each matchup, each

manager played half the games as the home team and half as the away team. Although the half game and pseudo-full game manager timed out or ran out of memory on some of the lineups, we were able to run at least 1000 simulated games between each manager. Table 2 shows the results of the games played by deep Lindsey. Deep Lindsey lost to both of its opponents about 49% of the time, making it our tournament's second runner-up.

<i>Deep Lindsey's Opponent</i>	<i>Games</i>	<i>Wins</i>	<i>Losses</i>	<i>Pct.</i>	<i>p value</i>
<i>Half Game</i>	1,675	862	813	0.515	0.120
<i>Pseudo-Full Game</i>	1,228	628	600	0.511	0.221

Table 2: The win and loss records of the half and pseudo-full game managers against deep Lindsey. Both managers beat deep Lindsey, and both won about 51% of the simulated games.

Finally, in 1,280 simulated games between the half game manager and the pseudo-full game manager, the pseudo-full game manager won 643 games and lost 637 for a win rate of 50.2% (p value: 0.44), making the half game manager a very close runner up. We declare the pseudo-full game manager to be the champion AI manager!

5. Cash, Boone, and the Margins of Victory

Having crowned the pseudo-full game manager as the champion AI manager, we now revisit Cash and Boone's infamous World Series decisions to see what actions the pseudo-full game manager would have taken. We begin with Kevin Cash's decision to relieve Blake Snell in Game Six of the 2020 World Series, which immediately backfired when Betts hit a double off of the relief pitcher and eventually scored the game winning run for the Dodgers. The 2020 Rays had a very talented bullpen, and Kevin Cash was known throughout the season to be quick to relieve a starting pitcher in favor of his bullpen, including Blake Snell. Snell had been extremely effective during the first five innings of the game, shutting out the Dodgers with nine strikeouts while allowing just one hit. However, after he allowed his second hit of the game with one out in the bottom of the sixth inning, Cash chose to bring in reliever Nick Anderson to face Mookie Betts, Corey Seager, and Justin Turner, who all struck out in their previous two at-bats against Snell. In defense of Kevin Cash, Anderson had been an excellent reliever for the Rays all season, and bringing in a good right-handed pitcher to face a good right-handed hitter like Betts rather than letting the left-handed Snell face him a third time seems like a reasonable strategy. It is a very tough decision, which is why the pseudo-full game manager is so valuable for decisions like this.

Unfortunately for Cash and the Rays, the pseudo-full game manager did not support the decision to relieve Blake Snell with Nick Anderson. The pseudo-full game manager estimated the expected win probability for the Rays if they chose to let Snell face Betts to be 0.584. Bringing in Anderson to face Betts gave a lower expected win probability of 0.554. Credit to Cash, of all the relievers he could have brought in, the pseudo-full game manager estimated that Anderson was the best option, and perhaps if our simple matchup model considered pitcher fatigue the pseudo-full game manager would have sided with Cash, but with just our simple matchup model we conclude that relieving Snell was the wrong decision.

Looking ahead to the 2024 World Series, we now evaluate Aaron Boone's tenth inning decisions in the Yankees loss in Game One to the Dodgers. The Yankees entered the bottom of the tenth inning with a 3 to 2 lead over the Dodgers. The bottom third of the Dodgers lineup were due up, and Boone began the inning by bringing in Yankees reliever Jake Cousins. This was an important decision because Shohei Ohtani, Mookie Betts, and Freddie Freeman, who at that time had four combined league MVP awards (Ohtani won NL MVP in 2024, so that total is now five), were due up fourth, fifth, and sixth in the inning. The Yankees certainly wanted to avoid having to face those three batters with a chance to win the game. Rather than bringing in Cousins, the pseudo-full game manager recommended bringing in Marcus Stroman. Stroman is typically a starting pitcher though, so even though he did not start any of the five games of the World Series, it is possible Boone was saving him for a later start. If we remove Stroman as an option, the pseudo-full game manager recommended bringing in Nestor Cortes, which gave the Yankees an expected win probability of 0.8034. In comparison, the pseudo-full game manager estimated that bringing in Cousins gave the Yankees an expected win probability of 0.8028, so Boone's decision to bring in Cousins was marginally suboptimal compared to bringing in Cortes. Unfortunately for the Yankees, Cousins retired just one of the batters he faced, so with Ohtani due up to bat with one out and runners on first and second base, Boone had to decide who he wanted to pitch to Shohei Ohtani. He chose Nestor Cortes, which the pseudo-full game manager deemed to be the optimal strategy. Many fans and media members felt that Tim Hill was the right choice for that moment in the game [16], but the pseudo-full game manager estimated that Cortes gave the Yankees an expected win probability of 0.669, whereas Tim Hill gave them an expected win probability of 0.627. Cortes retired Ohtani for the second out. He then intentionally walked Betts to load the bases before ultimately giving up the game-losing grand slam to Freddie Freeman.

Boone's decision to use Cousins instead of Cortes at the start of the tenth cost the Yankees an estimated 0.0006 in expected win probability, his decision to use Cortes instead of Hill to face Ohtani was optimal and cost the Yankees nothing in expected win probability, and his decision to intentionally walk Betts cost the Yankees 0.009 in expected win probability. Thus, in total Boone's decision-making in the tenth inning only cost the Yankees 0.0096 in expected win probability, which is less than 1% in an inning where they started with an 80% expected win probability. The decision to bring in Cortes over Hill to face Ohtani was primarily criticized because Cortes had not pitched in over a month due to injury. When asked about that decision, Boone said he "Just liked the matchup" [16], and the pseudo-full game manager agrees. It liked that matchup too. Overall, we conclude that although Boone was not perfect in the tenth inning, he made close to optimal decisions according to the pseudo-full game manager. We encourage Yankees fans to be more forgiving of Boone's Game One decisions.

6. Conclusion

The purpose of this paper was to create an AI baseball game manager to aid actual baseball managers with the numerous difficult decisions they have to make during a game. We modeled baseball as a stochastic, zero-sum, perfect-information, extensive-form game played between two managers who both want to maximize their win probability. We determined that the state space in this game is too large to solve for an optimal pair of managerial strategies, so instead we developed five AI managers that efficiently approximate an optimal pair of strategies. We showed that managers in actual MLB games who more closely followed our baseline AI manager, the Lindsey strategy, won a statistically significant majority of the time. Therefore, any AI manager that can beat

the baseline Lindsey strategy in simulated games is likely capable of aiding managers in their decision making, so we evaluated our other four AI managers by having them play simulated games against the baseline strategy and against each other. After running a tournament of simulated games, we determined that our pseudo-full game manager was the champion AI manager. We illustrated how this AI manager can help actual managers by revisiting the controversial decision made by Kevin Cash to relieve Blake Snell in Game 6 of the 2020 World Series and the controversial decision made by Aaron Boone to use Nestor Cortes in Game 1 of the 2024 World Series. We found that the decision to relieve Blake Snell was a suboptimal one, but the use of Nestor Cortes was optimal.

References

- [1] <https://www.retrosheet.org/>.
- [2] Baseball savant. <https://baseballsavant.mlb.com/>.
- [3] Statcast. <https://www.mlb.com/glossary/statcast>.
- [4] Three-batter minimum. <https://www.mlb.com/glossary/rules/three-batter-minimum>.
- [5] <https://www.mlb.com/gameday/>, 2023.
- [6] Jim Albert, Benjamin S Baumer, and Max Marchi. *Analyzing Baseball Data with R*. CRC Press, Aug 2024.
- [7] Michael Alcorn. (batter|pitcher)2vec: statistic-free talent modeling with neural player embeddings. In MIT Sloan Sports Analytics Conference. MIT Sloan, 2017.
- [8] Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015.
- [9] Woojin Doo and Heeyoung Kim. Modeling the probability of a batter/pitcher matchup event: A bayesian approach. *PLOS ONE*, 13(10):1–11, 10 2018.
- [10] Matt Haechral. Matchup probabilities in major league baseball. *SABR Baseball Research Journal*, 2014.
- [11] Glenn Healey. Modeling the probability of a strikeout for a batter/pitcher matchup. *IEEE Transactions on Knowledge and Data Engineering*, 27(9):2415–2423, 2015.
- [12] Bill James. *The Bill James baseball abstract*, 1983. Ballantine Books, New York, 1983.
- [13] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. In Johannes Furnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [14] James LeDoux. Introducing pybaseball: An open source package for baseball data analysis. <https://jamesrledoux.com/projects/open-source/introducing-pybaseball/>, Jul 2017.
- [15] George R. Lindsey. An investigation of strategies in baseball. *Operations Research*, 11(4):477–501, 1963.
- [16] Anthony McCarron. Yankees’ Nestor Cortes decision in World Series Game 1 loss is not as outrageous as it seems. <https://sny.tv/articles/yankees-aaron-boone-nestor-cortes-why-world-series-game-1>, Oct 2024.
- [17] Bill Petti and Saiem Gilani. *baseballr: Acquiring and Analyzing Baseball Data*, 2024. R package version 1.6.0, <https://github.com/BillPetti/baseballr>.
- [18] Scott Powers, Trevor Hastie, and Robert Tibshirani. Nuclear penalized multinomial regression with an application to predicting at bat outcomes in baseball. *Statistical Modelling*, 18(5-6):388–410, 2018.
- [19] Todd Roberts. *Mlb-statsapi*. <https://pypi.org/project/MLB-StatsAPI/>, Nov 2024.
- [20] Stuart Jonathan Russell and Peter Norvig. *Artificial intelligence : a modern approach*. Pearson, Cop, Boston, 2010.

- [21] Yoav Shoham and Kevin Leyton-Brown. Multiagent systems : algorithmic, game-theoretic, and logical foundations. Cambridge University Press, Cambridge ; New York, 2009.
- [22] Tom Tango. Marcel 2012. <https://www.tangotiger.net/archives/stud0346.shtml>, 2012.
- [23] Juan Toribio. Pulling Snell backfires on Cash: “Petty tough”. <https://www.mlb.com/news/blake-snell-world-series-game-6>, Oct 2020.